```
import kagglehub
import os
import glob

# Download the NEW dataset
path = kagglehub.dataset_download("gourangomandal/patient-data-healthcare-monitoring-system")

print("Path to dataset files:", path)

# Automatically find the CSV file in the downloaded folder
csv_files = glob.glob(path + "/*.csv")
csv_path = csv_files[0]
print("Found CSV:", csv_path)
```

```
Downloading to /root/.cache/kagglehub/datasets/gourangomandal/patient-data-healthcare-monitoring-system/1.archive...
100%|██████████| 744k/744k [00:00<00:00, 1.53MB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/gourangomandal/patient-data-healthcare-monitoring-system/versions/1
Found CSV: /root/.cache/kagglehub/datasets/gourangomandal/patient-data-healthcare-monitoring-system/versions/1/Synthetic_patient
```

```
import pandas as pd

# Load the new dataset
df = pd.read_csv(csv_path)

# Print columns just to be safe
print("Actual Columns in CSV:", df.columns.tolist())

# 1. Rename columns to match what our Web App expects
# (Using the regular '2' instead of subscript '₂')
df = df.rename(columns={
    'Heart Rate (bpm)': 'Heart_Rate',
    'SpO2 Level (%)': 'Oxygen_Saturation'
})

# 2. Create the missing "Risk_Level" target
# Logic: If both are NORMAL = Normal. One ABNORMAL = Medium. Both ABNORMAL = High.
def create_risk_level(row):
    hr_alert = row['Heart Rate Alert']
    spo2_alert = row['SpO2 Level Alert'] # Fixed to regular 2 here!

    if hr_alert == 'NORMAL' and spo2_alert == 'NORMAL':
        return 'Normal'
    elif hr_alert == 'ABNORMAL' and spo2_alert == 'ABNORMAL':
        return 'High'
    else:
        return 'Medium'

df['Risk_Level'] = df.apply(create_risk_level, axis=1)

# 3. Filter down to just our 3 required columns
filtered_df = df[['Heart_Rate', 'Oxygen_Saturation', 'Risk_Level']]

print("\n--- New Data Preview ---")
print(filtered_df.head())
print("\n--- Risk Level Counts ---")
print(filtered_df['Risk_Level'].value_counts())
```

```
Actual Columns in CSV: ['Patient Number', 'Heart Rate (bpm)', 'SpO2 Level (%)', 'Systolic Blood Pressure (mmHg)', 'Diastolic Blo

--- New Data Preview ---
   Heart_Rate  Oxygen_Saturation Risk_Level
0          98                 96     Normal
1         105                 97     Medium
2          90                 85     Medium
3         102                 87       High
4          81                 95     Normal

--- Risk Level Counts ---
Risk_Level
Medium    24392
Normal    23685
High      11923
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import joblib

le = LabelEncoder()
filtered_df['Risk_Level_Encoded'] = le.fit_transform(filtered_df['Risk_Level'])

X = filtered_df[['Heart_Rate', 'Oxygen_Saturation']]
y = filtered_df['Risk_Level_Encoded']

# The new dataset has 60,000 rows! We will train on a large batch.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print("--- Risk Level Mapping ---")
print(mapping)

joblib.dump(le, 'label_encoder.pkl')
print("\nSuccess: label_encoder.pkl saved!")
```

```
--- Risk Level Mapping ---
{'High': np.int64(0), 'Medium': np.int64(1), 'Normal': np.int64(2)}

Success: label_encoder.pkl saved!
/tmp/ipykernel_7694/475237031.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver
  filtered_df['Risk_Level_Encoded'] = le.fit_transform(filtered_df['Risk_Level'])
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import joblib

# Initialize and train
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Test
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"--- Model Accuracy: {accuracy * 100:.2f}% ---")
print("\n--- Detailed Performance Report ---")
print(classification_report(y_test, y_pred, target_names=le.classes_))

# Save the Brain
joblib.dump(model, 'pulse_ox_model.pkl')
print("\nSuccess: pulse_ox_model.pkl saved and ready for download!")
```

```
--- Model Accuracy: 100.00% ---

--- Detailed Performance Report ---
              precision    recall  f1-score   support

        High       1.00      1.00      1.00      2374
      Medium       1.00      1.00      1.00      4798
      Normal       1.00      1.00      1.00      4828

    accuracy                           1.00     12000
   macro avg       1.00      1.00      1.00     12000
weighted avg       1.00      1.00      1.00     12000


Success: pulse_ox_model.pkl saved and ready for download!
```

```
import pandas as pd
import joblib

# 1. Load your newly trained 60,000-row model
loaded_model = joblib.load('pulse_ox_model.pkl')
loaded_le = joblib.load('label_encoder.pkl')

def get_smart_prediction(bpm, spo2):
    # Fix the warning by providing exact feature names as a DataFrame
    input_df = pd.DataFrame([[bpm, spo2]], columns=['Heart_Rate', 'Oxygen_Saturation'])
```

```
    # Get prediction and calculate confidence percentage
    pred_num = loaded_model.predict(input_df)[0]
    prob = max(loaded_model.predict_proba(input_df)[0]) * 100
    risk = loaded_le.inverse_transform([pred_num])[0]

    # Updated Advice Logic for your new dataset
    advice = "Continue monitoring."
    if risk == "High":
        advice = "EMERGENCY: Immediate medical attention required."
    elif risk == "Medium":
        advice = "Abnormal: Rest and re-test in 5 minutes."
    elif risk == "Normal":
        advice = "Vitals are stable. You are doing great."

    return risk, prob, advice

# --- TEST IT OUT ---
test_bpm = 145
test_spo2 = 88
risk, cert, msg = get_smart_prediction(test_bpm, test_spo2)

print(f"--- Prediction Result ---")
print(f"Input: Heart Rate = {test_bpm} BPM, SpO2 = {test_spo2}%")
print(f"Predicted Risk: {risk} ({cert:.2f}% Confidence)")
print(f"Advice: {msg}")
```

```
--- Prediction Result ---
Input: Heart Rate = 145 BPM, SpO2 = 88%
Predicted Risk: High (100.00% Confidence)
Advice: EMERGENCY: Immediate medical attention required.
```

```
import pandas as pd
import joblib

# 1. Load your newly trained 60,000-row model
loaded_model = joblib.load('pulse_ox_model.pkl')
loaded_le = joblib.load('label_encoder.pkl')

def get_smart_prediction(bpm, spo2):
    # Format the input exactly how the Random Forest expects it
    input_df = pd.DataFrame([[bpm, spo2]], columns=['Heart_Rate', 'Oxygen_Saturation'])

    # Get prediction and calculate confidence percentage
    pred_num = loaded_model.predict(input_df)[0]
    prob = max(loaded_model.predict_proba(input_df)[0]) * 100
    risk = loaded_le.inverse_transform([pred_num])[0]

    # Updated Advice Logic (Matches your new dataset categories)
    advice = "Continue monitoring."
    if risk == "High":
        advice = "EMERGENCY: Immediate medical attention required."
    elif risk == "Medium":
        advice = "Abnormal: Rest and re-test in 5 minutes."
    elif risk == "Normal":
        advice = "Vitals are stable. You are doing great."

    return risk, prob, advice

# --- TEST IT OUT ---
# Try plugging in different numbers to see how your new model reacts!
hr_input = 145
spo2_input = 88
risk, cert, msg = get_smart_prediction(hr_input, spo2_input)

print(f"RESULT: {risk} Risk ({cert:.2f}% Confidence)")
print(f"ADVICE: {msg}")
```

```
RESULT: High Risk (100.00% Confidence)
ADVICE: EMERGENCY: Immediate medical attention required.
```